EasyShelf

Quick user guide



What it is

A floating panel of buttons over Blender's editors, Maya-shelf style. Every mode and editor has its own shelf with its own buttons: Object Mode, Edit Mode, Sculpt Mode, UV Editor, Shader Editor, Geometry Nodes. Each shelf can be shown as a panel, as a radial pie menu on a hotkey, or both. Each button runs its own Python code on click.

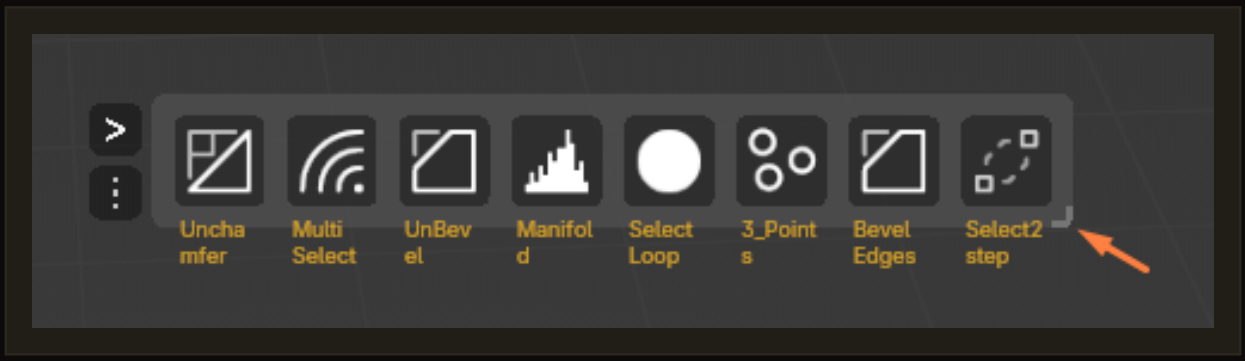
Orientation and moving the shelf



The ">" toggle and drag handle to the left of the panel

To the left of the panel: ">" toggles horizontal/vertical, and the three-dot handle drags with the mouse. Position is saved automatically.

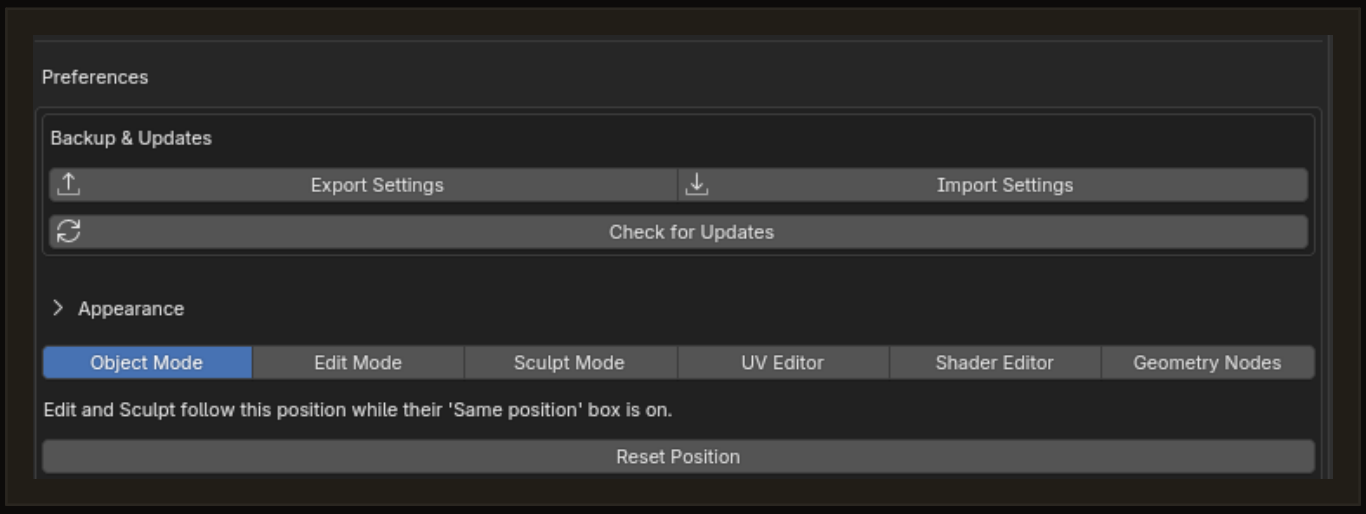
Shelf size



The resize corner (marked with an arrow)

A barely visible arc runs along the panel's bottom-right corner (bottom-left in vertical layout). It lights up on hover; hold the left mouse button and drag to resize the shelf: away from the panel's corner makes it larger, toward it smaller. The arc turns orange while you drag. The size is saved — it is the same Shelf Size value as the slider in the settings.

A shelf for every context



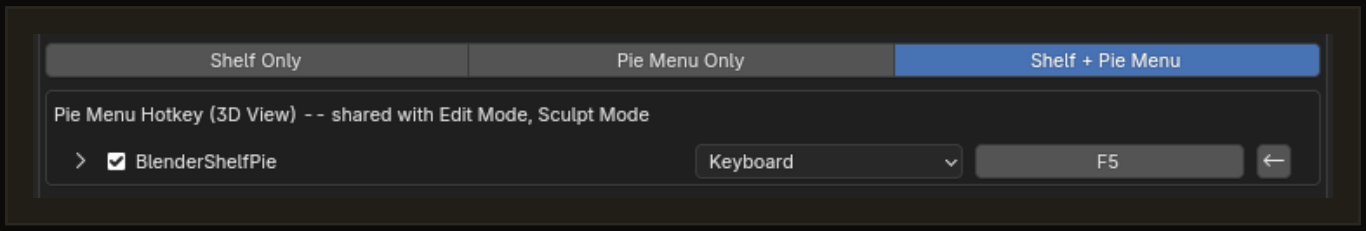
Context switcher, Enable, Reset Position

In Preferences, the context switcher chooses which shelf you are configuring: the main Shelf (Object Mode), or Sculpt, UV, Shader, Geometry Nodes. Each context has its own enable checkbox, button list, display mode and position.

For Edit Mode and Sculpt Mode, "Same position as Object Mode" is on by default, so the shelf does not jump when you switch modes in the 3D viewport. Untick it to place the shelf separately.

"Reset Position" puts the shelf back at the default spot, top center.

Display mode and hotkey



Shelf / Pie / Both mode and the pie hotkey

Each context has a mode: Shelf (panel only), Pie (pie menu only) or Both.

The pie menu always mirrors the shelf of the current context: in Edit Mode it shows the Edit shelf's buttons, in the UV Editor the UV shelf's. Button order matches the order in the settings list.

The hotkey is assigned per editor: the 3D viewport (shared by Object, Edit and Sculpt Mode), the UV Editor, and the Node Editor (shared by the Shader Editor and Geometry Nodes). You can use the same hotkey for every pie menu, or assign different ones.

The pie menu in the viewport



The EasyShelf pie menu (Object Mode)

The assigned hotkey opens the radial menu with the buttons of the shelf whose context it was called in.

Preferences: general settings

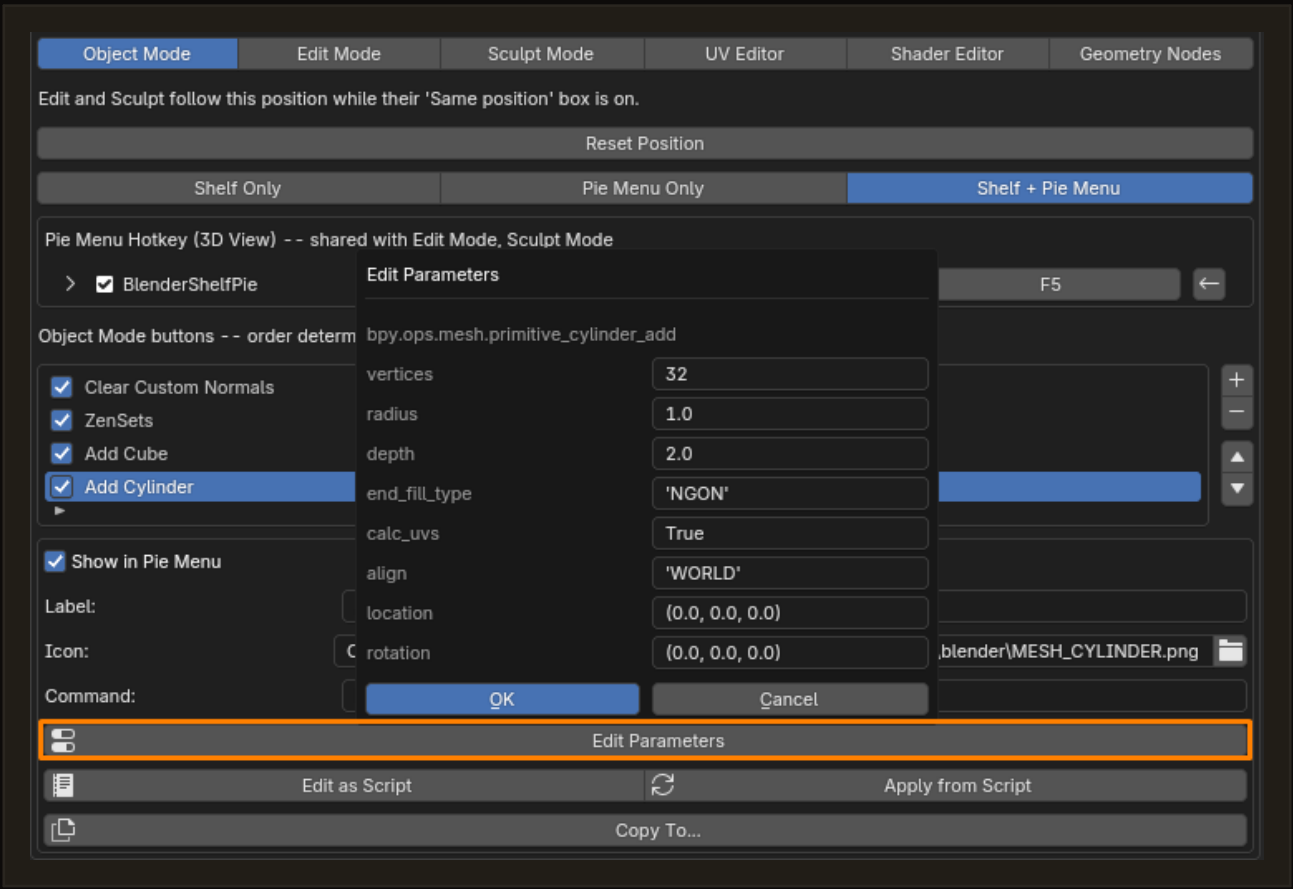
The "Backup & Updates" box: Export/Import Settings save and load every addon setting as one JSON file, and "Check for Updates" checks the website for a newer release and offers a download link.

The collapsible "Appearance" panel applies to every shelf: button size and icon opacity, labels (visibility, font size, color, placement relative to the button), button and background colors, and Show Number for ordinal numbers.

The button list

Below the context settings is that context's button list. Every button has its own card: label, icon, command. The arrows reorder, "+" and "−" add and remove buttons, and the Show in Pie checkbox leaves a button out of the pie menu.

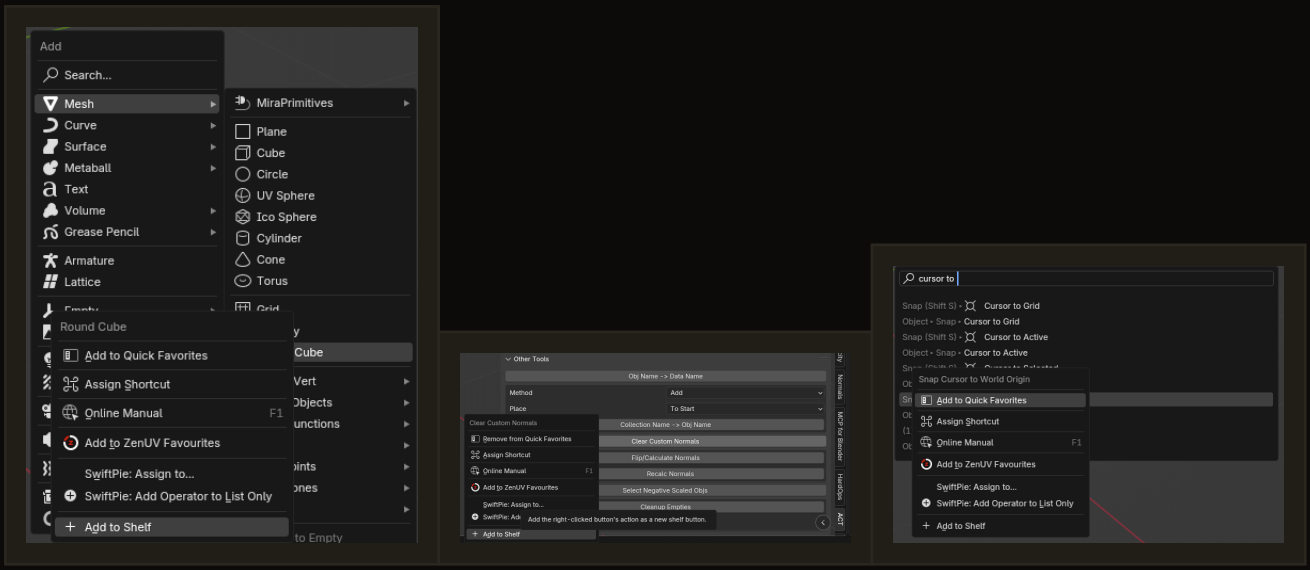
Editing parameters without Python



Edit Parameters: tweaking a cylinder's parameters without Python

"Edit Parameters" opens a window where you can edit the parameters a function exposes — for example a cylinder's vertex count, radius and depth — without writing or editing any Python.

Ways to add a function (button) to a shelf



"Add to Shelf" works from any button's context menu: a built-in operator, a third-party addon button, a search result (F3)

1. "+" in the settings list — an empty placeholder, type the command and label yourself.

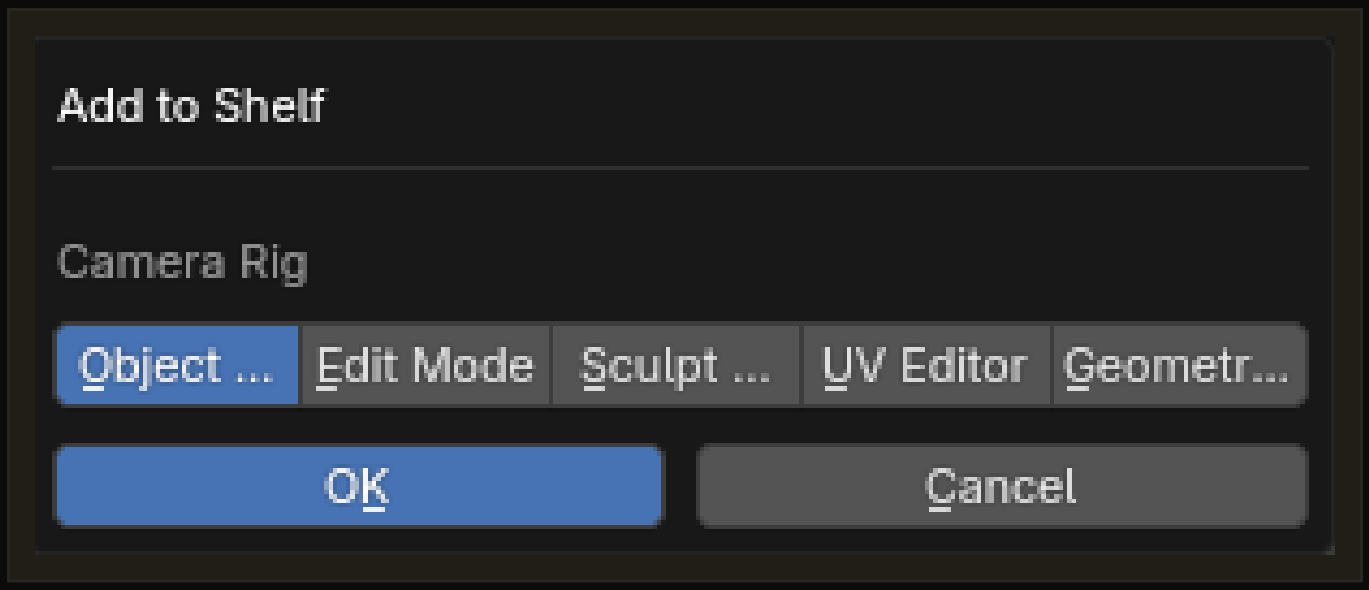
2. Right-click any interface button, including third-party addons → "Add to Shelf".

3. Find the function through operator search (F3), then likewise right-click → "Add to Shelf".

4. Right-click a shelf button → "Add Script": an empty button and a Text Editor to paste a script into.

5. Right-click a shelf button → "Import Script (.py)": a button from an existing Python file (details in "A script on a button").

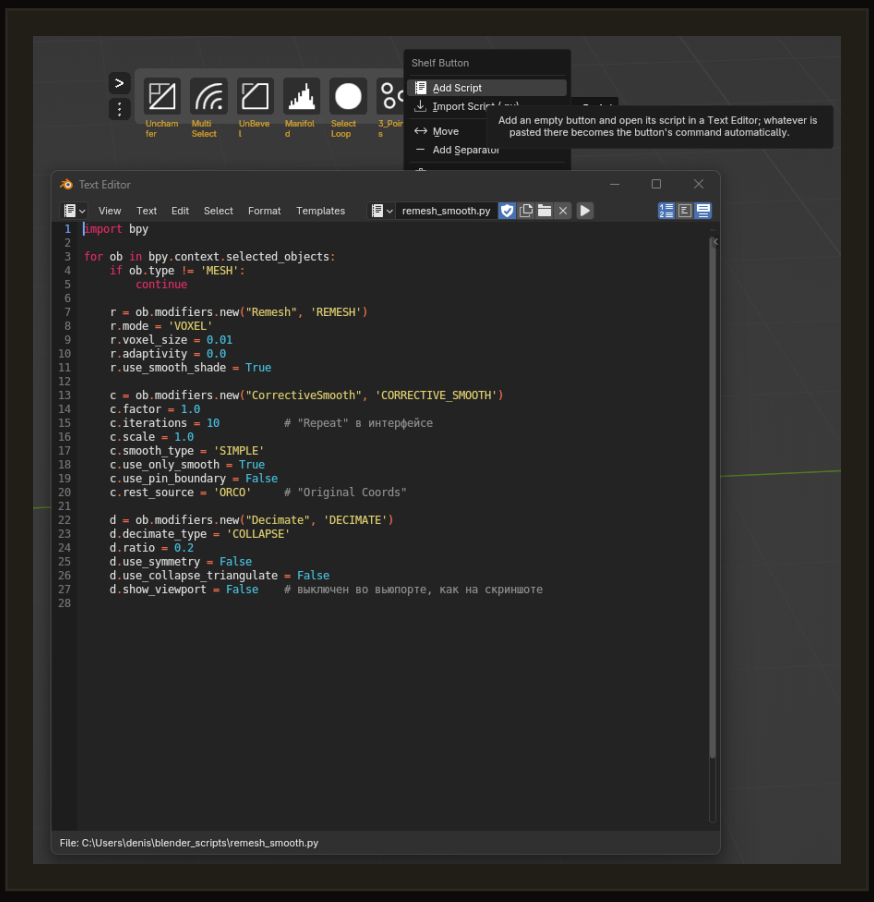
Choosing a shelf when adding



The "Add to Shelf" dialog: pick the shelf that receives the button

If several shelves are enabled, "Add to Shelf" opens a dialog. It already suggests the shelf that fits the current editor and mode.

A script on a button



"Add Script": the menu item and the Text Editor where the code is pasted

A button can run a whole Python script, not just one command. Right-click a shelf button to get two new menu items.

You can write the script yourself or ask an AI assistant to write it: describe what the button should do and paste the code.

"Add Script" creates an empty button with the default icon and opens its script in a Text Editor. Paste your code there — a moment later it is written into the button's command automatically, no extra click needed.

"Import Script (.py)" creates a button from an existing Python file, named after the file.

The icon is picked automatically: from a "# icon: NAME" tag in the first lines of the script, from the first bpy.ops call, or from the name. If nothing fits, the default stays and you can change it by hand.

The script is copied into the button. If you edit the file on disk afterwards, the button does not update — import it again.

Icons

Icons are picked automatically for popular tools: the addon ships its own icons for UV operations (Unwrap, Pack, Stitch, Gridify, Straighten, Export Layout and more) and for the main sculpt brushes (Draw, Clay Strips, Grab, Smooth, Inflate and more). If nothing fits, choose your own with the folder button next to the Icon field.

Copying functions between shelves

The "Copy To..." button on a function's card copies it to another enabled shelf — for example from Object to Edit. Editing the command in one list does not touch the copy in the other.

What to put in Command

You can add your own script or macro (a sequence of commands), written yourself or with an AI assistant (Claude, ChatGPT, etc.). For longer scripts there are "Edit as Script" and "Apply Script": the code is edited in Blender's Text Editor. For quickly tweaking a single operator call, "Edit Parameters" is easier — no Python editing needed.